



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Case 3

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus
04/11/2025

Abstract

Implementation of a man-in-the-middle attack with an attacker (Kali Linux) and a victim (Ubuntu). The attacker positions itself between the victim and the gateway using arpspoofing to monitor the connection. We use the packet sniffer from Case 2 to record traffic into a .pcap file. The capture is analyzed with Wireshark to draw conclusions.

Contents

Introduction

Practical part of the assignment

Bibliography

Introduction

For this exercise, which is a continuation of the previous one, we will need to use one more operating system in VMware. We will choose Ubuntu 24.04.3 as the victim and Kali Linux as the attacker. On both operating systems we must have the same Bridge network interface type, so that they are on the same network and can communicate with each other. In addition, the NAT network adapter provides IPs to the virtual machines through the host's virtual network, whereas in Bridge mode the IPs are given directly by the router. Although we had initially started with NAT, we noticed that the victim's connection to the gateway/virtual router was interrupted immediately after using arpspoof, and for this reason we redid the exercise with Bridge.

Practical part of the assignment

Introduction:

First, we will identify the IP addresses on both systems, and then, with the nmap tool, we will confirm that the Ubuntu IP corresponds to the victim. Then, with the arpspoof tool, we will position ourselves as the man-in-the-middle in the communication between Ubuntu and the router: arpspoof will send fake ARP replies so that the victim 'sees' the MAC Address of the gateway as the MAC Address of Kali Linux, and the gateway 'sees' the MAC Address of the victim as the MAC Address of Kali Linux. We intend not to interrupt the connection between the victim and the router,

but simply to monitor the traffic exchanged between them. Once we complete the process with arpspoof, we will enable promiscuous mode and run the sniffer from Case 2 so that it records the packets into a .pcap file. After the capture is finished, we will open the .pcap file with the Wireshark tool for analysis and to draw conclusions.

Starting the procedures:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ ifconfig
br-480f97c7b15a: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 02:42:b8:64:a7:dc txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ba:30:70:88 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.109 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::2a02:1388:4003:265d:94f8:efb5:9279:c05c prefixlen 64 scopeid 0x0<global>
    ether 00:0c:29:fe:e8:b5 txqueuelen 1000 (Ethernet)
    RX packets 590 bytes 55776 (54.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 296 bytes 31781 (31.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

We observe that the attacker's IP (Kali Linux) is 192.168.1.109 with the ifconfig command.

```
kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 172.20.10.4/23 brd 172.20.10.255 scope global noprefixroute ens32
        valid_lft forever preferred_lft forever
    inet 192.168.1.108/24 brd 192.168.1.255 scope global dynamic noprefixroute ens32
        valid_lft 86053sec preferred_lft 86053sec
kalaitzon@ubuntudesktop1:~$ ip route
default via 192.168.1.1 dev ens32 proto dhcp src 192.168.1.108 metric 100
172.20.10.0/23 dev ens32 proto kernel scope link src 172.20.10.4 metric 100
192.168.1.0/24 dev ens32 proto kernel scope link src 192.168.1.108 metric 100
kalaitzon@ubuntudesktop1:~$
```

We also observe on Ubuntu (the victim) that its IP is 192.168.1.108 with the ip a command, and that the gateway/router IP is 192.168.1.1 with the ip route command.

Now, with the nmap tool, we will scan the subnet and see which IPs are active (among which is the victim's). This is done with the command `sudo nmap -sn 192.168.1.0/24`. The reason we use /24 is to 'catch' every active IP of the network mask (which range from .0 to .254) and display them to us.

```
(kali@kali)-[~]
$ nmap -sn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:27 EST
Nmap scan report for 192.168.1.1
Host is up (0.019s latency)
MAC Address: [redacted] (TP-Link Limited) router
Nmap scan report for 192.168.1.103
Host is up (0.020s latency)
MAC Address: [redacted] device
Nmap scan report for 192.168.1.107
Host is up (0.0013s latency)
MAC Address: [redacted] device Technology Singapore PTE.)
Nmap scan report for 192.168.1.108
Host is up (0.0050s latency)
MAC Address: 00:0C:29:F5:51:1F (VMware)
Stats: 0:00:11 elapsed; 255 hosts completed (4 up), 255 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Stats: 0:00:15 elapsed; 255 hosts completed (4 up), 255 undergoing Host Discovery
Parallel DNS resolution of 1 host. Timing: About 0.00% done
Nmap scan report for 192.168.1.109
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 15.16 seconds
```

From the scan (`nmap -sn 192.168.1.0/24`) we observe that the address 192.168.1.1 is the gateway/router. The address 192.168.1.109 corresponds to the attacker machine (Kali Linux), as we know from `ifconfig`, while 192.168.1.108 is the victim's (Ubuntu), because the remaining IPs belong to my devices. So we know the victim's IP with certainty. However, for confirmation, we can also cross-check it via the MAC Address. With the command `sudo arp-scan --localnet` we can see which IP address corresponds to its own MAC Address. We can cross-check it with the ip a command we used above on Ubuntu.

```
(kali@kali)-[~]
$ sudo arp-scan --localnet
Interface: eth0, type: EN10MB, MAC: 00:0c:29:fe:e8:b5, IPv4: 192.168.1.109
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1 [redacted] (Unknown)
192.168.1.101 [redacted] (Unknown)
192.168.1.103 [redacted] (Unknown)
192.168.1.107 [redacted] (Unknown)
192.168.1.108 00:0c:29:f5:51:1f (Unknown)
192.168.1.100 [redacted] (Unknown: locally administered)
192.168.1.101 [redacted] (Unknown) (DUP: 2)
```

Under other circumstances this still does not hold, because as an attacker I do not know anything about the OS. With the command `nmap -vv -O -sV -Pn 192.168.1.108` (where -vv is for verbose, -O to detect the OS, -sV to detect a service, and -Pn so as not to ping this address since we know it is active), I would normally get the information that the OS is Ubuntu. But because it is the most recent version with the latest updates, nmap does not correctly identify it and we are not given this necessary information, as shown in the image below. All it confirms is that this machine with this IP is an OS, but it cannot recognise anything further (Too many fingerprints match this host to give specific OS details). However, in the case where we know the other IPs (that they belong to other devices), as now, it is sufficient.


```

(kali@kali)-[~]
$ nmap -vv -O -sV -Pn 192.168.1.108
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:30 EST
NSE: Loaded 47 scripts for scanning.
Initiating ARP Ping Scan at 10:30
Scanning 192.168.1.108 [1 port]
Completed ARP Ping Scan at 10:30, 0.09s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 10:30
Completed Parallel DNS resolution of 1 host. at 10:31, 6.62s elapsed
Initiating SYN Stealth Scan at 10:31
Scanning 192.168.1.108 [1000 ports]
Completed SYN Stealth Scan at 10:31, 0.27s elapsed (1000 total ports)
Initiating Service scan at 10:31
Initiating OS detection (try #1) against 192.168.1.108
Retrying OS detection (try #2) against 192.168.1.108
NSE: Script scanning 192.168.1.108.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
Nmap scan report for 192.168.1.108
Host is up, received arp-response (0.0020s latency).
Scanned at 2025-11-03 10:31:03 EST for 1s
All 1000 scanned ports on 192.168.1.108 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 00:0C:29:F5:51:1F (VMware)
Too many fingerprints match this host to give specific OS details
TCP/IP fingerprint:
SCAN(V=7.95%E=4%D=11/3%OT=CT=1%CU=43309%PV=Y%DS=1%DC=D%G=N%M=000C29%TM=6908CAB9%P=x86_64-pc-linux-gnu)
SEQ(CI=Z%II=I)
T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)
U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)
IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 1 hop

Read data files from: /usr/share/nmap

```

Next, we pinged each IP address to verify basic connectivity (reachability) before proceeding to arpspoofing from Kali Linux. With this we confirm that the IP addresses are up and that there is routing to each IP.

```

(kali@kali)-[~]
$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=6.91 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=1.93 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=1.72 ms
64 bytes from 192.168.1.108: icmp_seq=4 ttl=64 time=1.61 ms
^C
— 192.168.1.108 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 1.608/3.040/6.910/2.236 ms

(kali@kali)-[~]
$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=0.131 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=0.123 ms
^C
— 192.168.1.109 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.123/0.126/0.131/0.003 ms

(kali@kali)-[~]
$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=22.1 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=11.7 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.37 ms
^C
— 192.168.1.1 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 8.368/14.029/22.052/5.830 ms

```

We did the same from Ubuntu, as shown in the image below.

```
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=4.38 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=1.96 ms
^C
--- 192.168.1.109 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.242/2.525/4.379/1.342 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=0.108 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=0.110 ms
^C
--- 192.168.1.108 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2069ms
rtt min/avg/max/mdev = 0.068/0.095/0.110/0.019 ms
kalaitzon@ubuntudesktop1:~$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=91.3 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=7.74 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.61 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 7.738/35.890/91.327/39.201 ms
kalaitzon@ubuntudesktop1:~$
```

```
(kali@kali)-[~]
$ sudo ip link set dev
[sudo] password for kali
Sorry, try again.
[sudo] password for kali
```

```
(kali@kali)-[~]
$ ip link show eth0
```

```
2: eth0: <BROADCAST,MULTICAST,PRONISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff
```

We enable promiscuous mode with the command `sudo ip link set dev eth0 promisc on` and check it with the command `ip link show eth0`, in order to receive all the interface's traffic and analyse packets.

```
(kali@kali)-[~]
$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1

(kali@kali)-[~]
$ cat /proc/sys/net/ipv4/ip_forward
1
```

Next, we will write the value 1 to the system file `/proc/sys/net/ipv4/ip_forward` (with the command `echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward`). In a MITM and arpspoof scenario, if the attacker (Kali Linux) 'takes' the traffic between the victim and the gateway, it must forward the packets to the real gateway so that the victim continues to have Internet. Without IP forwarding, the attacker would receive the packets but would not forward them, resulting in the victim losing connection. We then checked the value with `cat /proc/sys/net/ipv4/ip_forward`, which returned 1, meaning that the `net.ipv4.ip_forward` parameter is enabled. Since it returns the value 1, it has been enabled.

The above concerns only IPv4, because arpspoof works via the ARP (Address Resolution Protocol) protocol and affects exclusively IPv4 traffic. ARP is a protocol that translates an IPv4 address into a physical network address. It is used so that a

Useful commands for arpspoof:

sudo pkill arpspoof | closes all arpspoof processes

ps aux | grep arpspoof | checks whether there are still processes with this name

Next, we compile the packet sniffer located in the Downloads folder. Once compilation is complete (gcc pcap_sniffer.c -o pcap_sniffer -lpcap), we run the sniffer on the eth0 interface as we did in Case 2. The packet capture will start and will stop as soon as we press Ctrl + C on the keyboard. The packets will be saved to the .pcap file, which we will open with Wireshark to extract the results.

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ cd Downloads

(kali@kali)-[~/Downloads]
$ ls -l *.c
-rw-rw-r-- 1 kali kali 2554 Oct 31 11:41 pcap_sniffer.c

(kali@kali)-[~/Downloads]
$ gcc pcap_sniffer.c -o pcap_sniffer -lpcap

(kali@kali)-[~/Downloads]
$ sudo ./pcap_sniffer eth0
[sudo] password for kali:
Capturing packets on eth0 ... Press Ctrl+C to stop.
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (82 bytes)
Captured packet (412 bytes)
Captured packet (102 bytes)
Captured packet (444 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (118 bytes)
Captured packet (110 bytes)
Captured packet (110 bytes)
Captured packet (60 bytes)
Captured packet (446 bytes)
Captured packet (466 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
```

```
(kali@kali)-[~/Downloads]
$ wireshark capture_libpcap.pcap
** (wireshark:39307) 11:24:34.647491
emPalette
** (wireshark:39307) 11:24:34.652403
ButtonPalette
** (wireshark:39307) 11:24:34.652673
onPalette
** (wireshark:39307) 11:24:34.652783
kBoxPalette
** (wireshark:39307) 11:24:34.652818
oButtonPalette
```

```
:::Palette) const QPlatformTheme::Syst
:::Palette) const QPlatformTheme::Tool
:::Palette) const QPlatformTheme::Butt
:::Palette) const QPlatformTheme::Chec
:::Palette) const QPlatformTheme::Radi
```

Opening the file capture_libpcap.pcap with the command (wireshark capture_libpcap.pcap) in Wireshark, we located and recorded the packet traffic between the victim (Ubuntu) and the gateway. In the capture, packets and requests appear towards various websites and services we visited during the test (e.g. Wikipedia, YouTube, Reddit), confirming the successful recording of the network activity.

capture_libpcap.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip.addr == 192.168.1.108

No.	Time	Source	Destination	Protocol	Length	Info
41	14.788558	192.168.1.108	185.125.190.56	NTP	90	NTP Version 4, client
62	21.120023	192.168.1.108	192.168.1.1	DNS	106	Standard query 0x4c02 A content-signature-2.cdn.mozilla.r
66	22.453744	192.168.1.108	192.168.1.1	DNS	95	Standard query 0xd018 A detectportal.firefox.com OPT
69	23.997411	192.168.1.108	192.168.1.1	DNS	86	Standard query 0xc491 A ads.mozilla.org OPT
71	24.724407	192.168.1.108	192.168.1.1	DNS	108	Standard query 0x6b5e A firefox.settings.services.mozilla
72	25.038156	192.168.1.108	185.125.190.58	NTP	90	NTP Version 4, client
75	25.672029	192.168.1.108	192.168.1.1	DNS	86	Standard query 0xf1b1 HTTPS www.youtube.com OPT
76	25.672121	192.168.1.108	192.168.1.1	DNS	86	Standard query 0xf7c5 A www.youtube.com OPT
77	25.672189	192.168.1.108	192.168.1.1	DNS	87	Standard query 0x3053 HTTPS en.wikipedia.org OPT
78	25.673204	192.168.1.108	192.168.1.1	DNS	87	Standard query 0x429b A en.wikipedia.org OPT
79	25.673715	192.168.1.108	192.168.1.1	DNS	88	Standard query 0x8df4 HTTPS www.wikipedia.org OPT
80	25.674887	192.168.1.108	192.168.1.1	DNS	85	Standard query 0x2a21 HTTPS www.reddit.com OPT
81	25.676070	192.168.1.108	192.168.1.1	DNS	85	Standard query 0xa424 A www.reddit.com OPT
82	25.676097	192.168.1.108	192.168.1.1	DNS	88	Standard query 0x3683 A www.wikipedia.org OPT
83	25.676469	192.168.1.108	192.168.1.1	DNS	89	Standard query 0x8b8c A addons.mozilla.org OPT
84	25.677732	192.168.1.108	192.168.1.1	DNS	89	Standard query 0x6de0 HTTPS addons.mozilla.org OPT

> Frame 41: 90 bytes on wire (720 bits), 90 bytes captured (720 bits) on interface 0
 > Ethernet II, Src: VMware_f5:51:1f (00:0c:29:f5:51:1f), Dst: VMware_f5:51:1f (00:0c:29:f5:51:1f)
 > Internet Protocol Version 4, Src: 192.168.1.108, Dst: 185.125.190.56
 > User Datagram Protocol, Src Port: 52355, Dst Port: 123
 > Network Time Protocol (NTP Version 4, client)

The image below, which comes from the analysis of the .pcap file, successfully records the victim's (Ubuntu) internet activity: calls to Wikipedia and YouTube appear as conversations to the respective domains/hosts.

Source: <https://apackets.com/>

Conclusion:

Through the exercise, it was demonstrated in practice how a Man in the Middle (MITM) attack can be carried out using the arpspoof tool, which deceives the victim and the gateway so that their communication passes through the attacker. By enabling IP forwarding and simultaneously recording the network traffic with Wireshark, it became possible to monitor the packets exchanged between them. Overall, the exercise highlighted the operation of the arpspoof tool and confirmed the ability to intercept and monitor packets at the local network (LAN) level.

Bibliography

danscourses. (2012, March 27). *ARP Spoofing - Man-in-the-middle attack* [Video file]. Retrieved from https://www.youtube.com/watch?v=hI9J_tnNDCc

Narten, T., Nordmark, E., & Simpson, W. (1998). *Neighbor Discovery for IP Version 6 (IPv6)*. <https://doi.org/10.17487/rfc2461>

Introduction. (n.d.). Bettercap. <https://www.bettercap.org/project/introduction/>

Tools of the Trade: IPv6 DNS Takeover with MitM6. (2023, September 22). <https://www.evolvesecurity.com/blog-posts/tools-of-the-trade-ipv6-dns-takeover-with-mitm6>

Kampourakis, V., Kambourakis, G., Chatzoglou, E., & Zaroliagis, C. (2022). Revisiting man-in-the-middle attacks against HTTPS. *Network Security*, 2022(3). [https://doi.org/10.12968/s1353-4858\(22\)70028-1](https://doi.org/10.12968/s1353-4858(22)70028-1)

Vaishnavi. (2025, June 19). How to Use Bettercap for Network Penetration Testing ? Beginner's Guide with Commands and Use Cases. *WebAsha Technologies*. <https://www.webasha.com/blog/how-to-use-bettercap-for-network-penetration-testing-beginners-guide-with-commands-and-use-cases>

GeeksforGeeks. (2025, October 10). *Address Resolution Protocol ARP*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/arp-protocol/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>

Kumar, G. (2025, July 15). *IPv6 Neighbor Discovery Protocol (NDP) explained*. <https://www.uninets.com/UniNets>. <https://www.uninets.com/blog/neighbor-discovery-protocol>

Infosec. (2021, April 27). *How to set up a man in the middle attack | Free Cyber Work Applied series* [Video]. YouTube. <https://www.youtube.com/watch?v=GkexkyUbUd4>

Tech Sky - Ethical Hacking. (2024, September 15). *How to Spy on Any Device's Network using ARP Spoofing in Kali Linux?* [Video]. YouTube. <https://www.youtube.com/watch?v=YtLsI-wHv2U>

CertBros. (2021, April 6). *ARP Poisoning | Man-in-the-Middle attack* [Video]. YouTube. <https://www.youtube.com/watch?v=A7nih6SANYs>

Chris Greer. (2021, December 9). *How ARP poisoning works // Man-in-the-Middle* [Video]. YouTube. <https://www.youtube.com/watch?v=cVTUeEoJgEg>